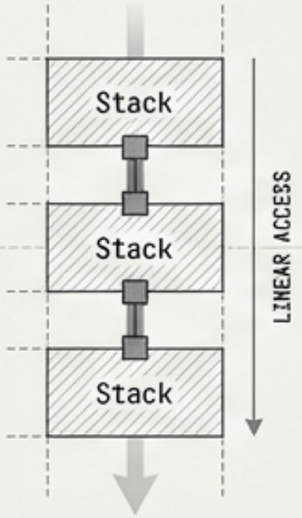
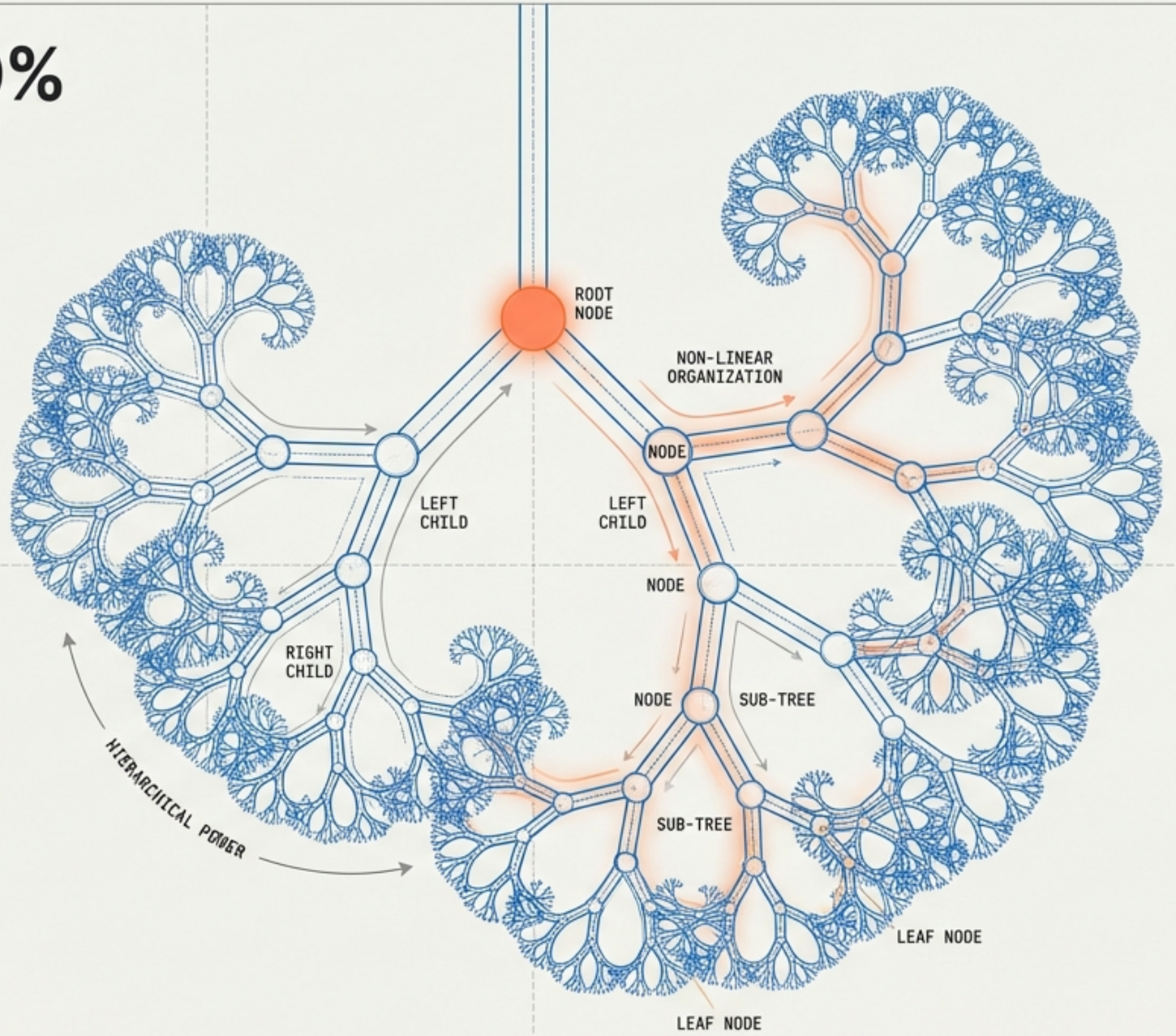
Mastery of Trees is 50% of Your Interview

Moving from Linear Constraints to Hierarchical Power.





THE STAKES: If you command the Tree, you crack the interview.

THE STAKES: If you command the Tree, you crack the interview.

THE SHIFT: Transitioning from linear sequential access to non-linear hierarchical organization.

THE GOAL: A comprehensive guide from anatomy to memory representation.

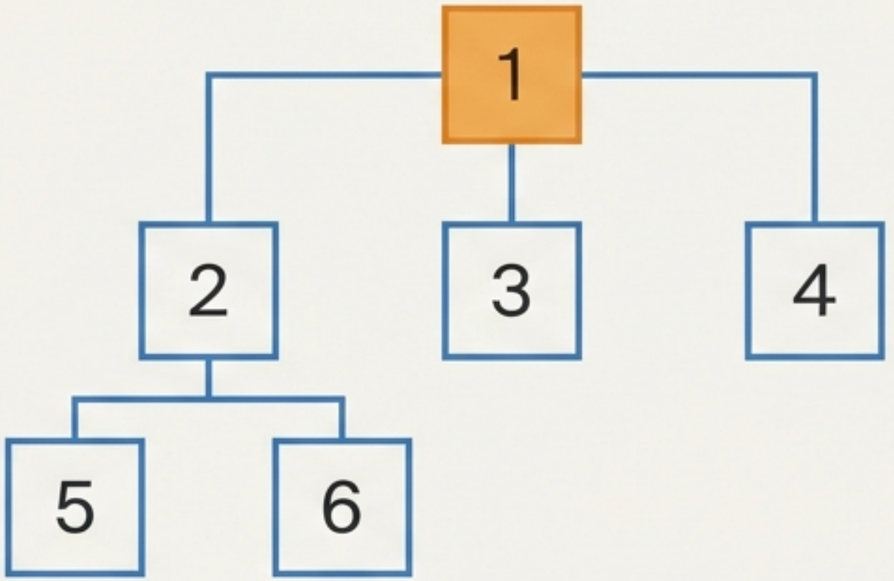
TITLE: TREE STRUCTURE NASTERY	SCALE: 1:1	DATE: OCT 26, 2024
DRAWN BY: ARCHITECTURAL BLUEPRINT & SWISS F		

LINEARITY VS. HIERARCHY

LINEAR (Array/Stack)

Sequential Access: Single path forward.

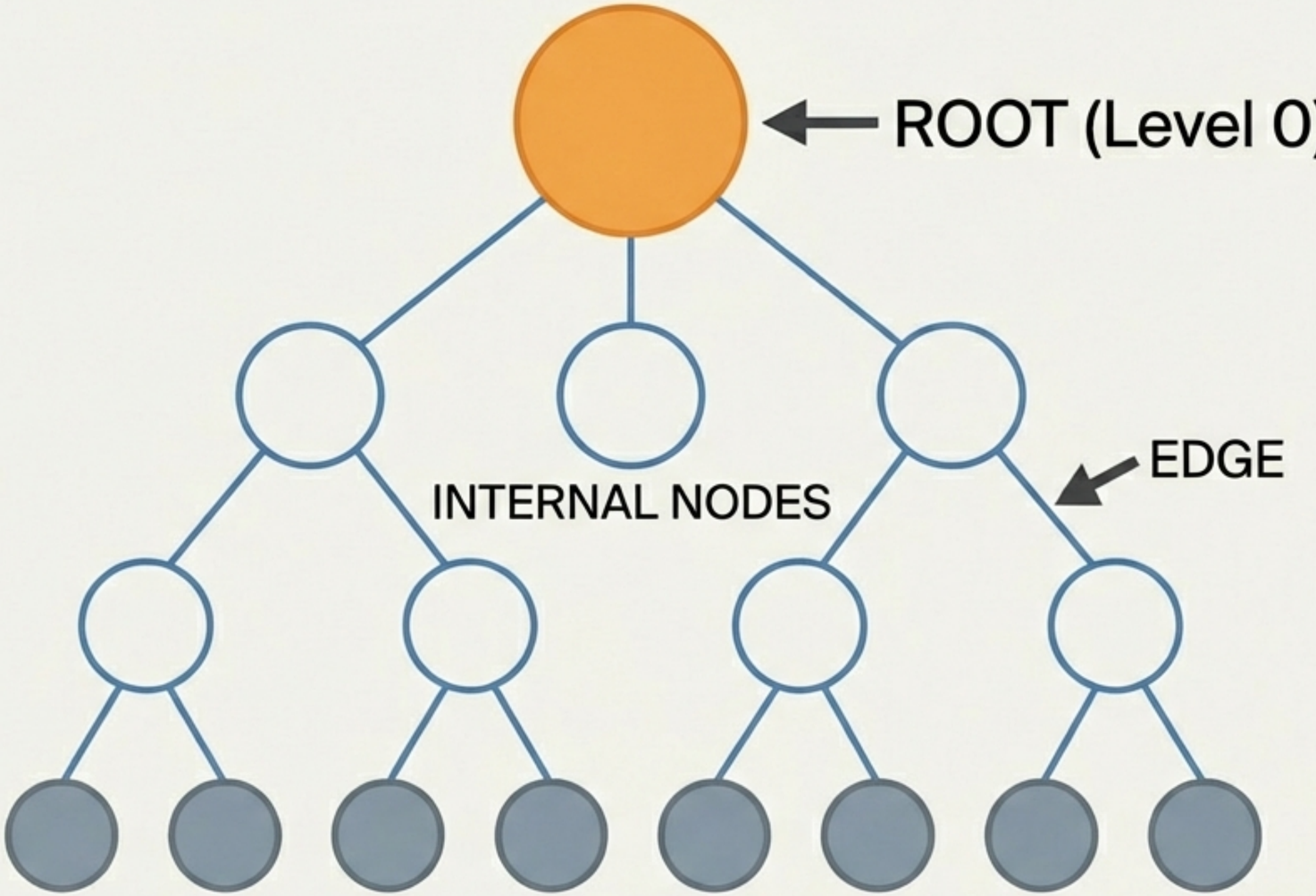
NON-LINEAR (Tree)



Hierarchical Access: Multiple options at every step.

- Linear structures (Arrays) restrict you to sequential access.
- Trees offer choices. You aren't forced to go 1→2; you can go 1→4 or 1→3.
- Ideal for organizing data with inherent levels (e.g., File Systems, Org Charts).

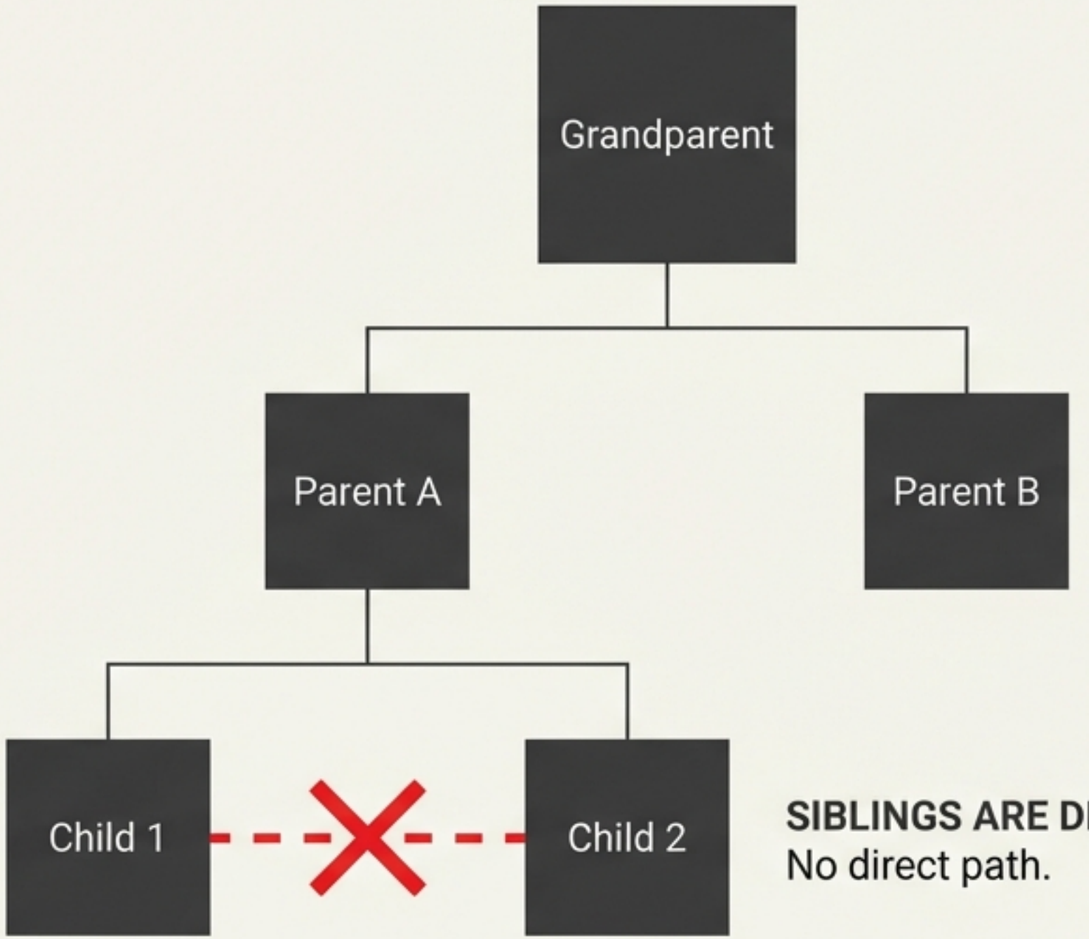
THE INVERTED ANATOMY



- **INVERTED REALITY:** In nature, roots are at the bottom. In CS, we rotate 180°.
- Root is the starting memory block.
- Leaves are dead ends.

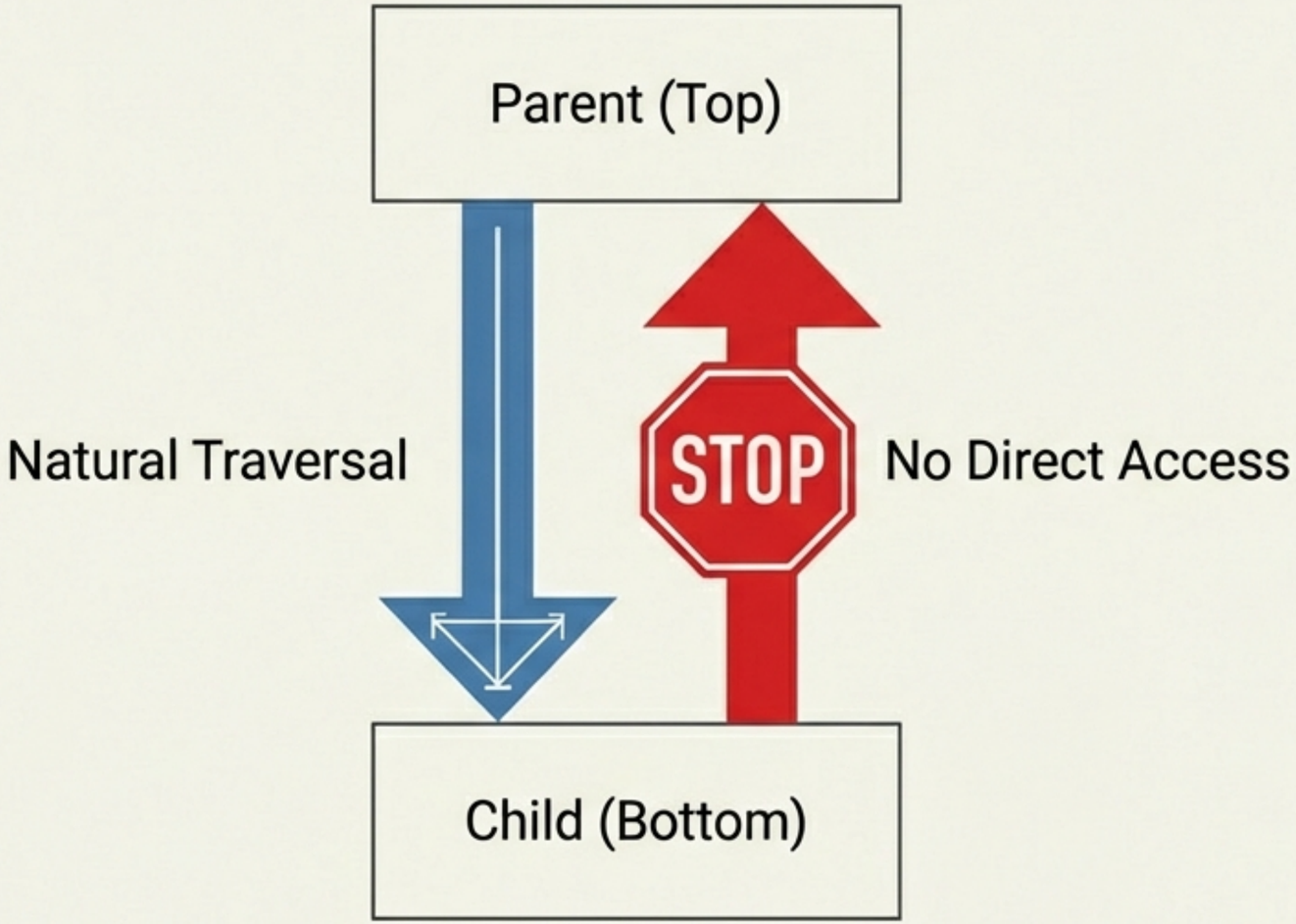
The Family Model & Connectivity Rules

- The Rule: To move between siblings, you must traverse up to the Parent and back down.
- Hierarchy Levels:
  Level 0 (Root),
  Level 1 (Children),
  Level 2 (Grandchildren).



SIBLINGS ARE DISCONNECTED.
No direct path.

The One-Way Street: Why We Need Recursion

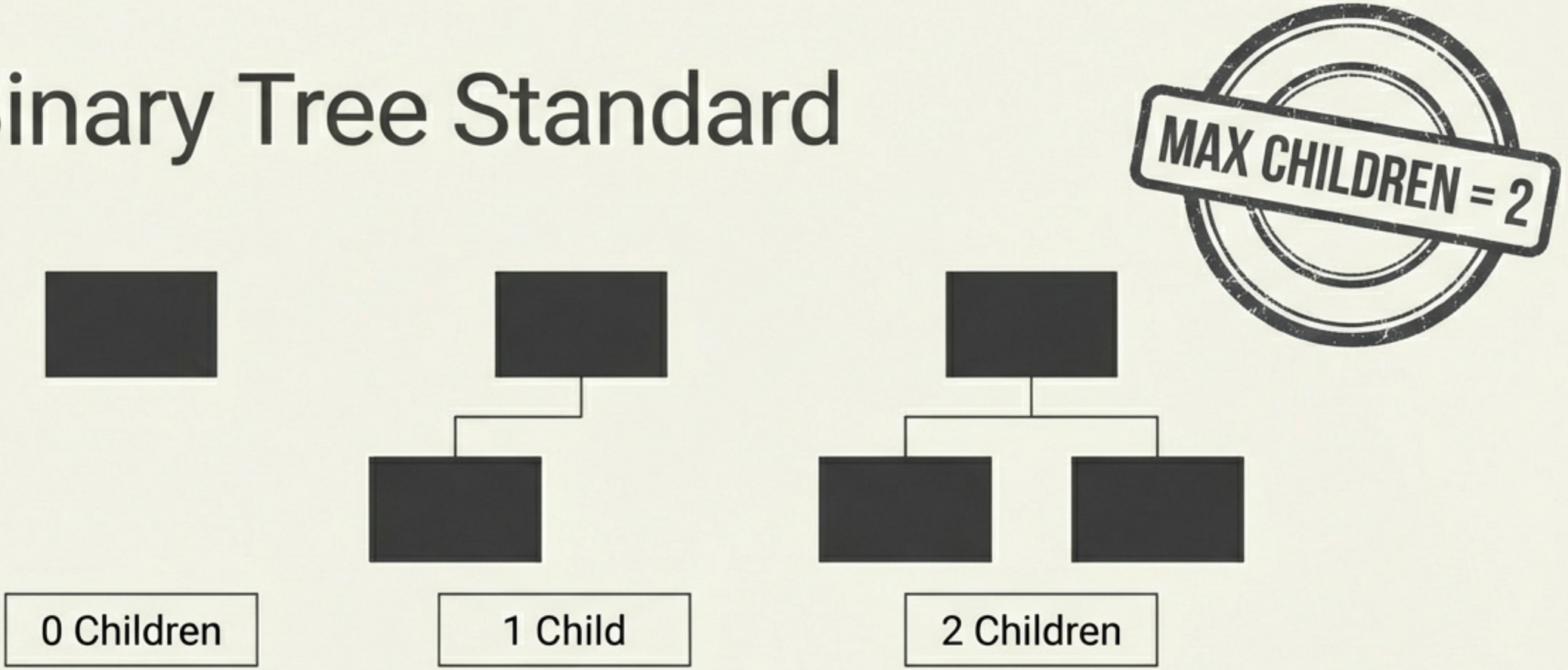


Constraint: You can traverse Top-to-Bottom freely. You cannot traverse Bottom-to-Top physically.

The Solution: RECURSION.

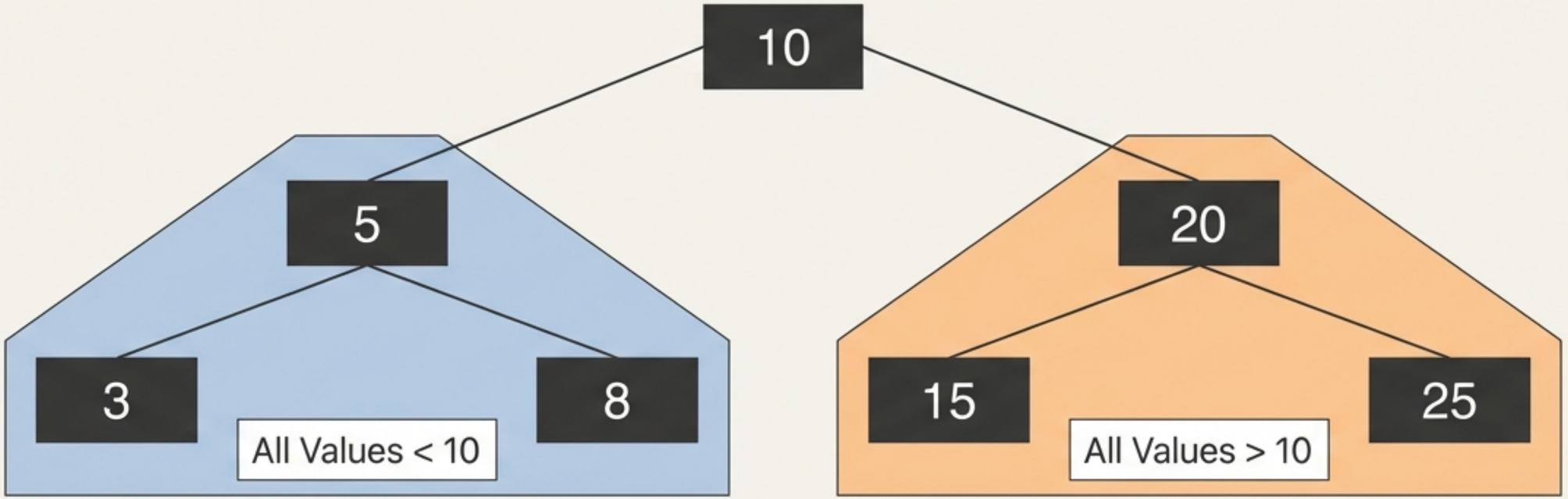
Insight: Recursion serves as the stack trace that allows logical backtracking where the physical structure prevents it.

The Binary Tree Standard



- **Definition**: A tree where every node has a maximum of 2 children.
- **Valid counts**: 0, 1, or 2.
- **Note**: This is the structural skeleton for 50% of interview questions.

Binary Search Tree (BST): Order in Chaos



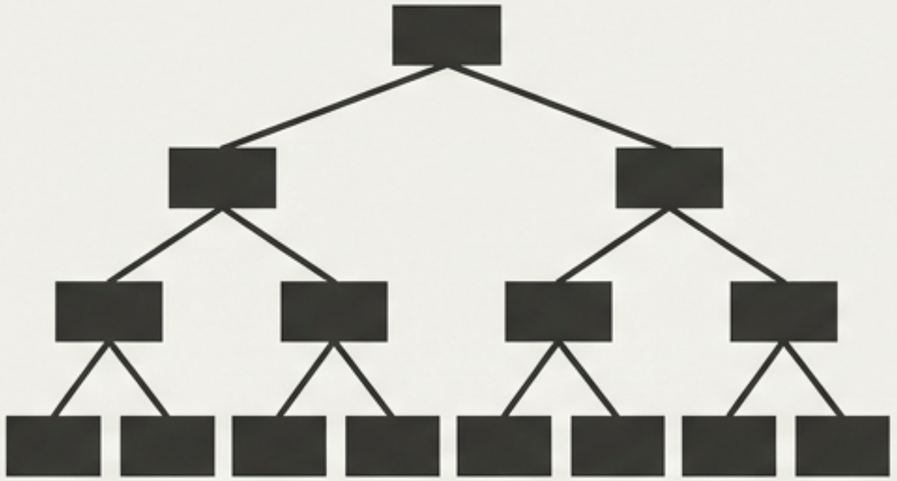
- The **Core Rule**: Left Subtree < Root < Right Subtree.
- **Crucial**: This applies to **ALL descendants**, not just the immediate child.

Trap:

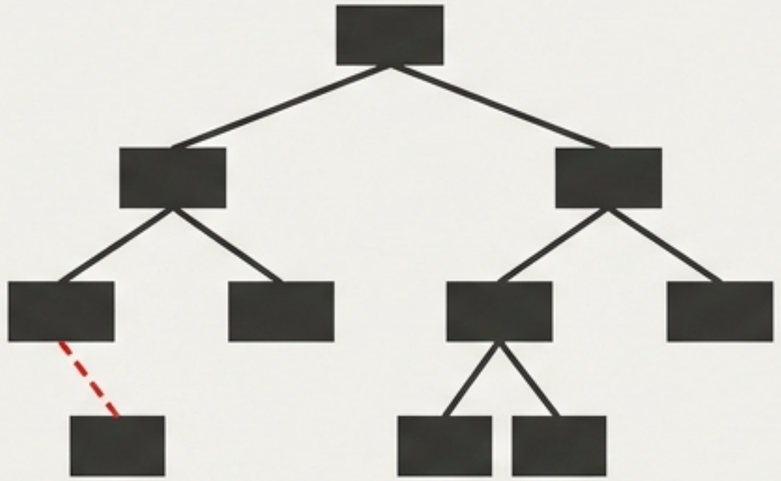
12 is > 10, so it CANNOT be in the left subtree.

Strict (Full) Binary Tree

The “All or Nothing” Constraint



VALID: 0 or 2 Children

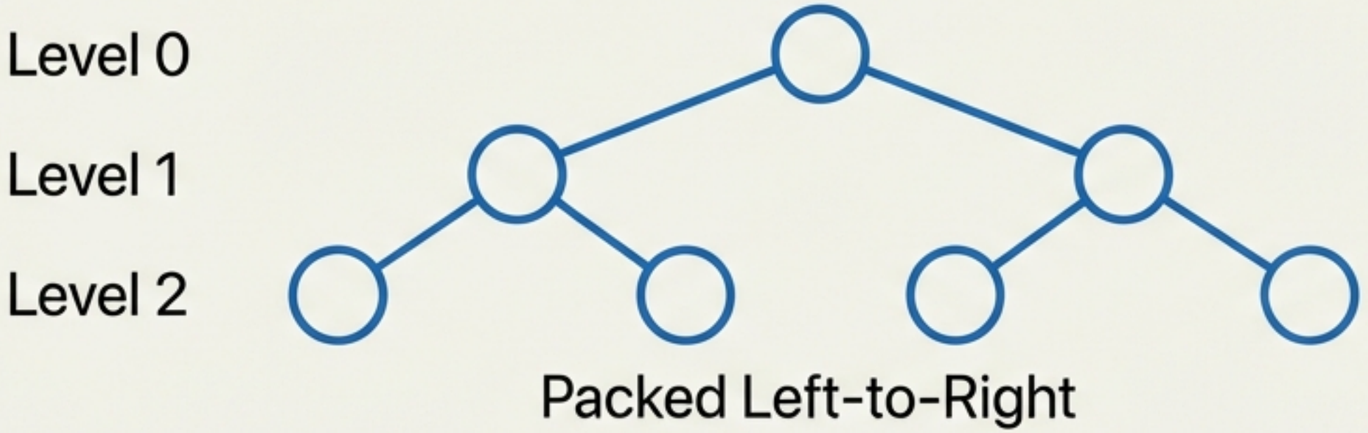


INVALID: Single Child Prohibited

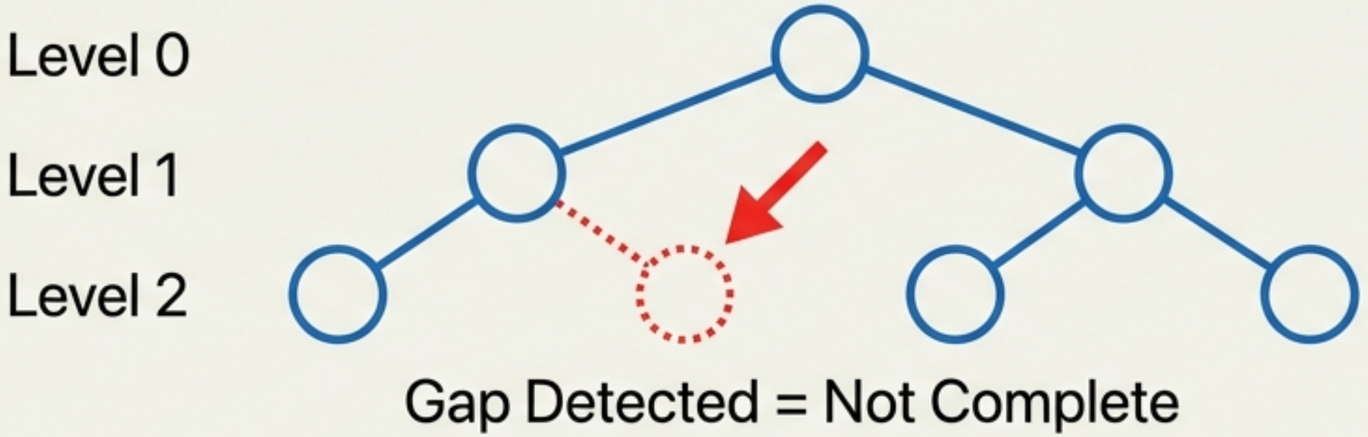
- Also known as a Full Binary Tree.
- The Rule: A node can have 0 children (Leaf) or 2 children. It can never have exactly 1.
- Analogy: Children are born as twins or not at all.

Complete Binary Tree (CBT)

The “Tetris” Rule: Filling the Gaps



Packed Left-to-Right



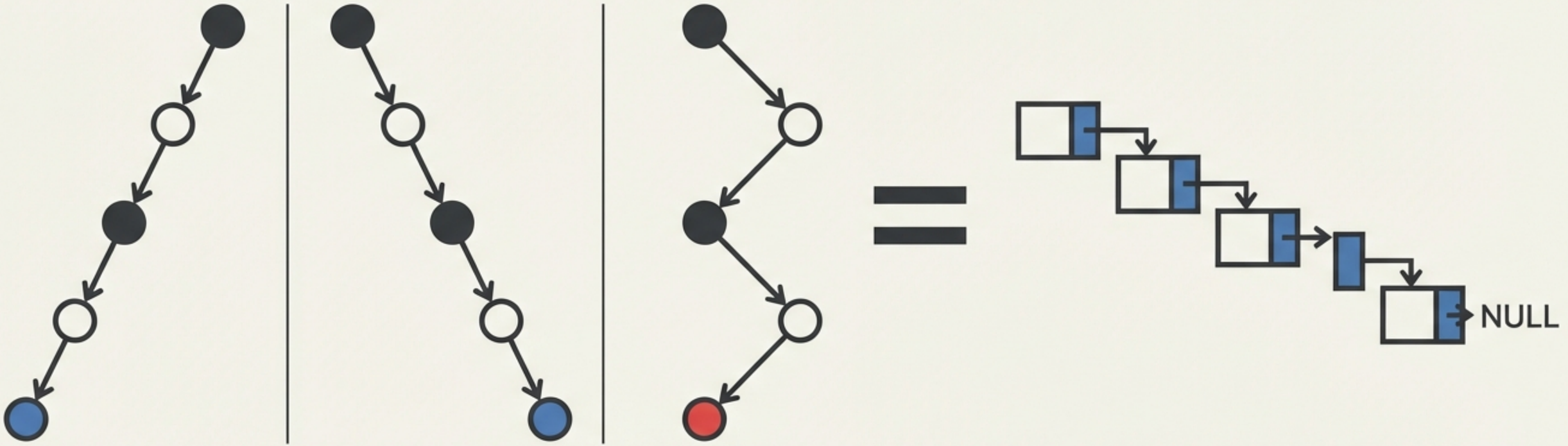
Gap Detected = Not Complete

The Rule: You cannot start a new level until the previous level is 100% full.

Packing: Nodes must be added as far must be added as far left as possible. No gaps allowed in the sequence.

Edge Cases: When Trees Fail

The "All or Nothing" Constraint



Left Skewed

Right Skewed

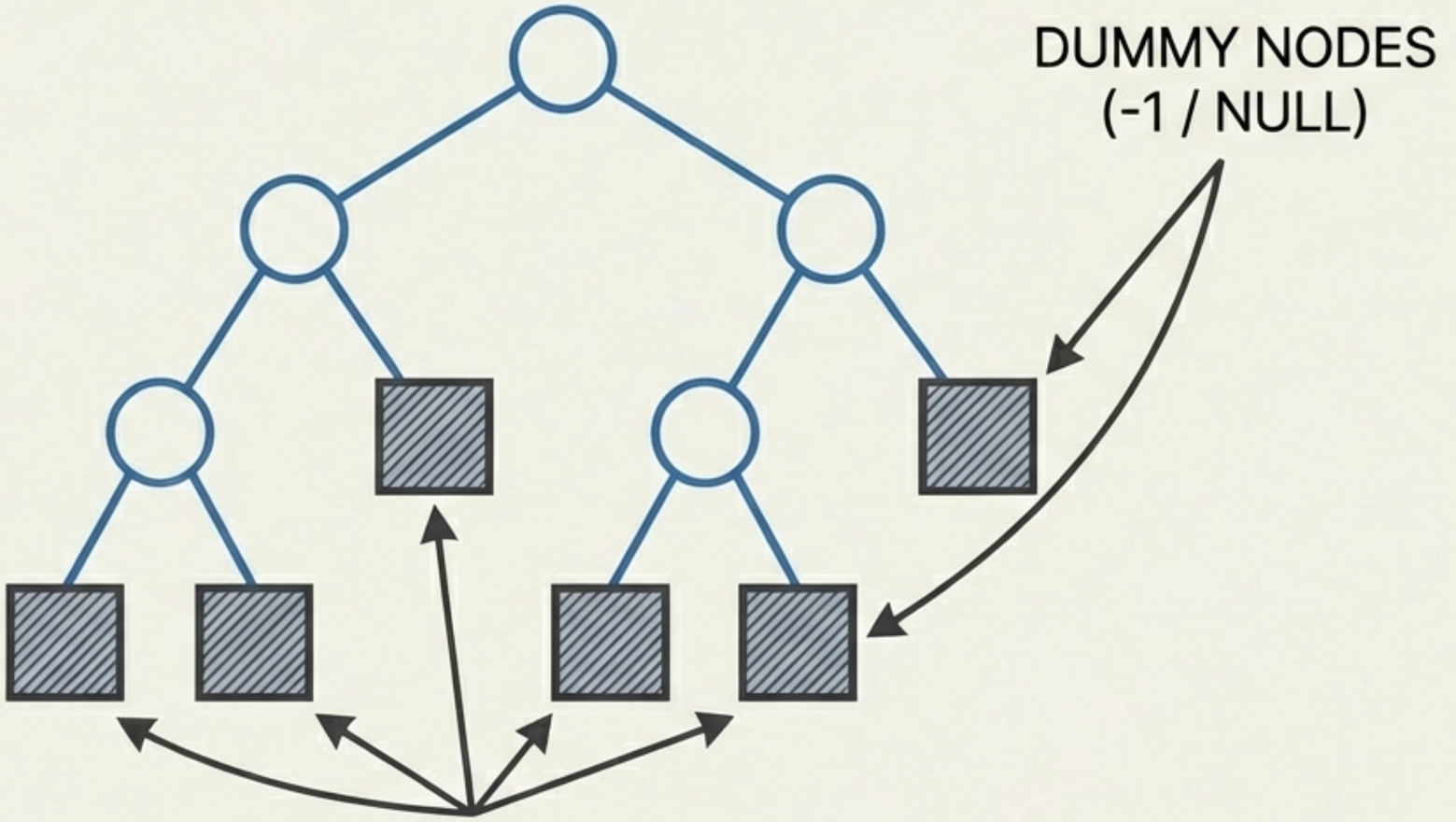
Degenerate / Zig-Zag

The Reality: These are technically trees, but they function as Linked Lists (O(n) performance). Every internal node has exactly one child, destroying the hierarchical advantage.

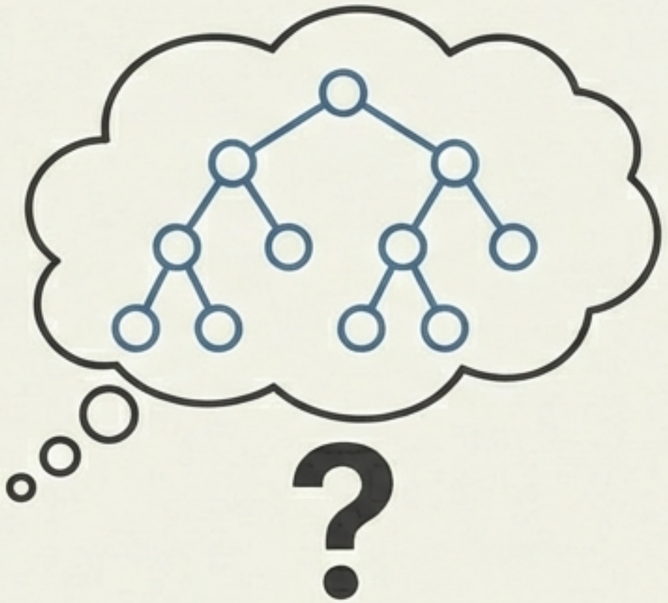
Extended Binary Tree

Concept: Converting a standard tree into a Strict/Full tree by filling empty spaces with Dummy Nodes.

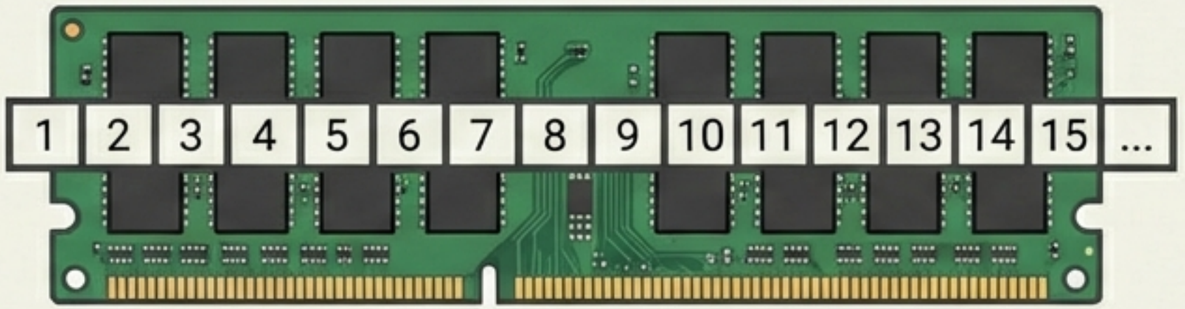
Purpose: Essential for visualizing empty spaces when mapping a tree to a fixed-size Array.



The Representation Dilemma



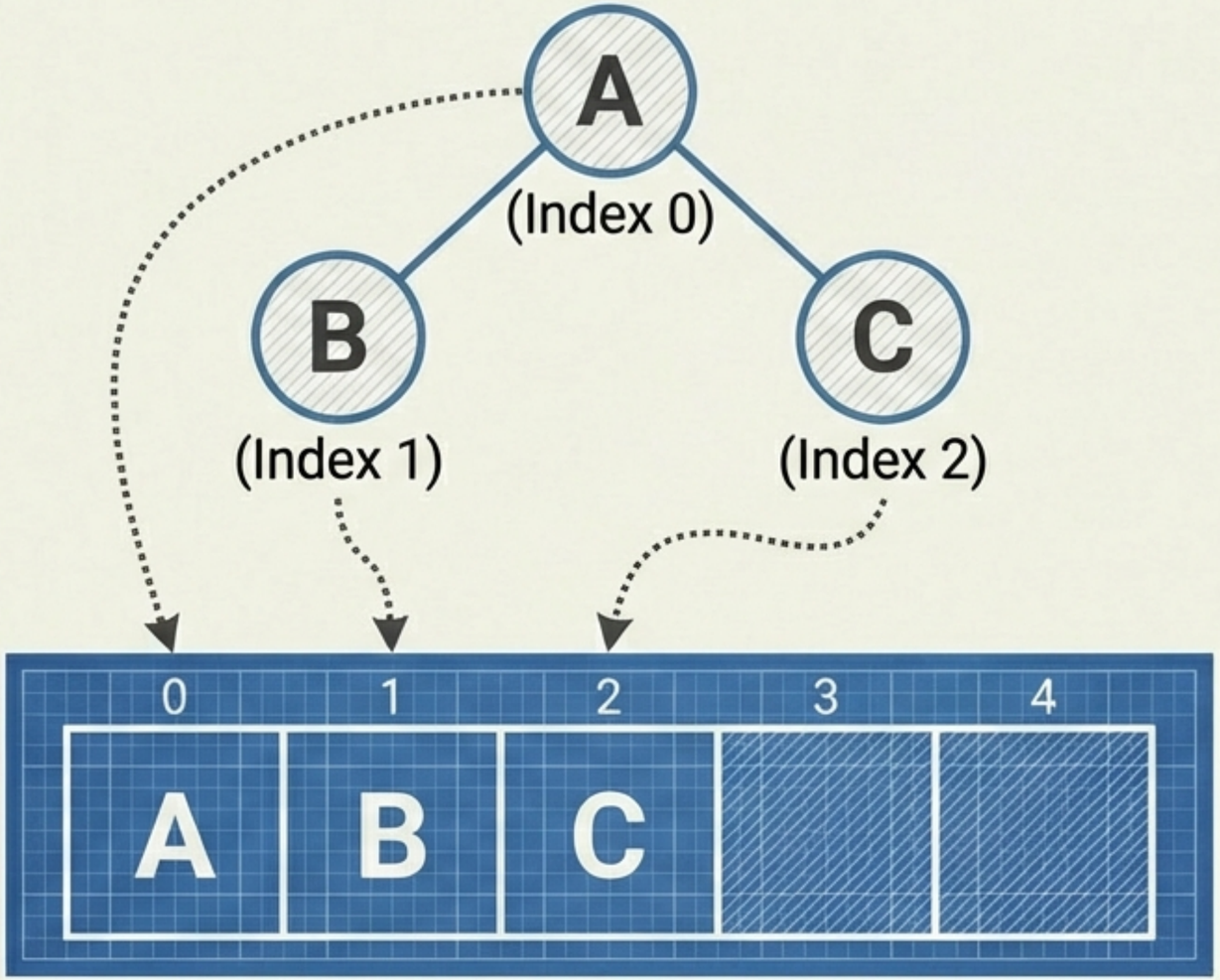
HOW WE IMAGINE IT
(Logical)



HOW MEMORY
STORES IT
(Physical)

The Challenge: Mapping a 2D hierarchical structure into 1D linear memory.
The Options: 1. Array (Sequential) vs. 2. Linked List (Dynamic).

Array Representation: The Math of Storage

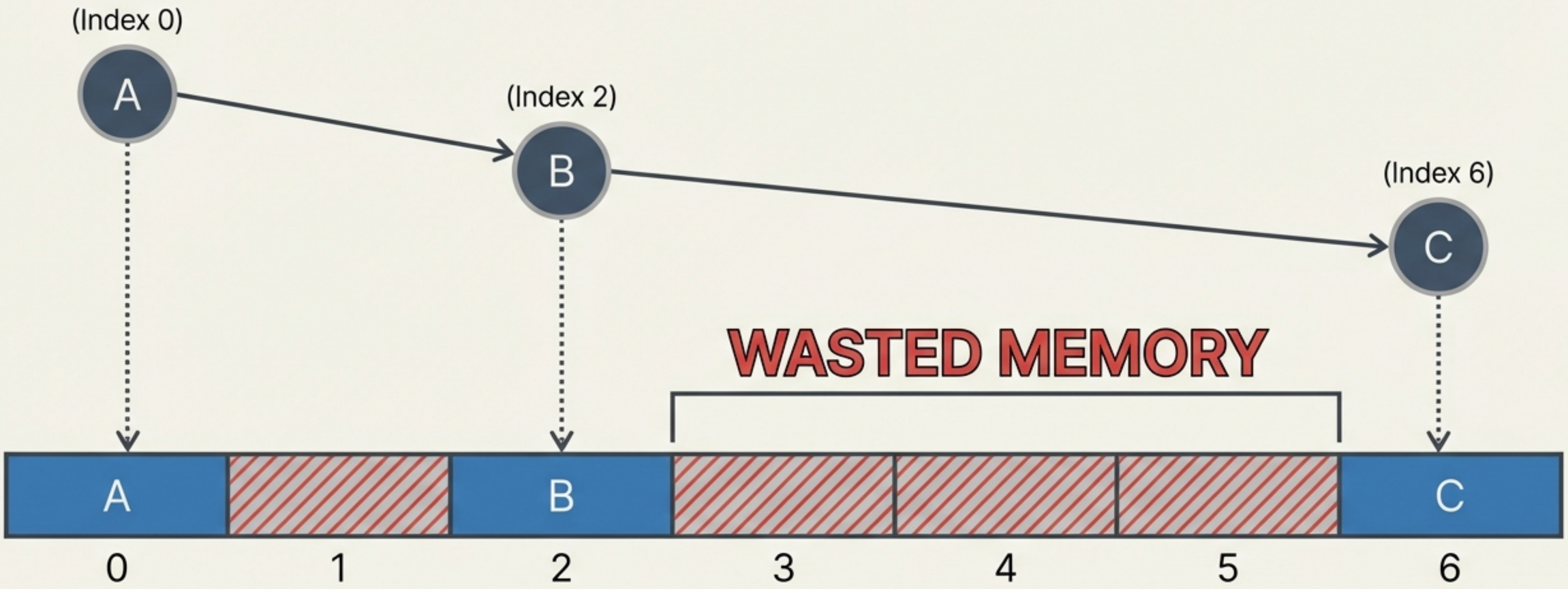


Index Mapping (if Root is 0):

Left Child = **2 * i + 1**
Right Child = **2 * i + 2**
Parent = **(i - 1) / 2**

Math defines the position.
No pointers needed.

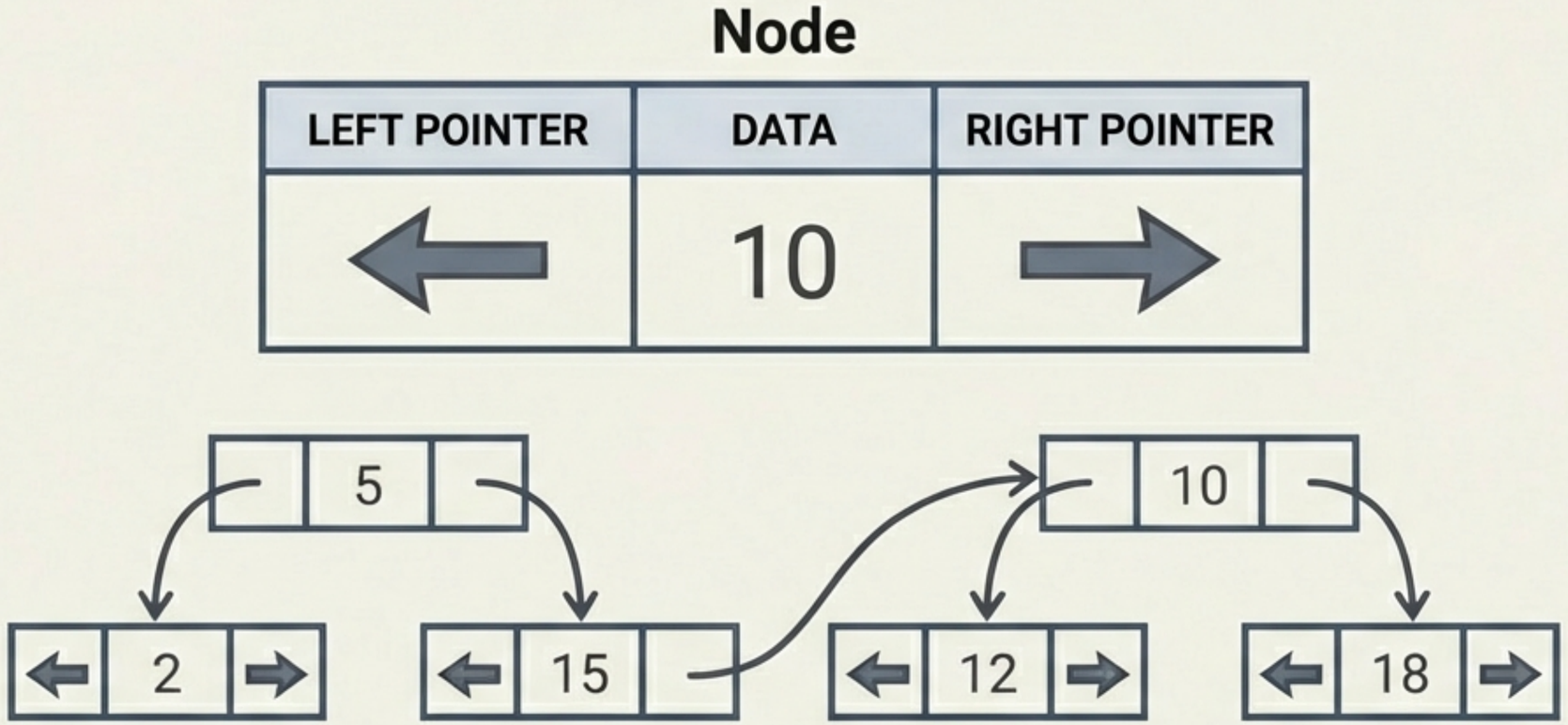
The Array Drawback: Memory Bloat



The Problem: To maintain the index math ($2i+2), you must reserve space for missing nodes.

Verdict: Arrays are inefficient for sparse or unbalanced trees.

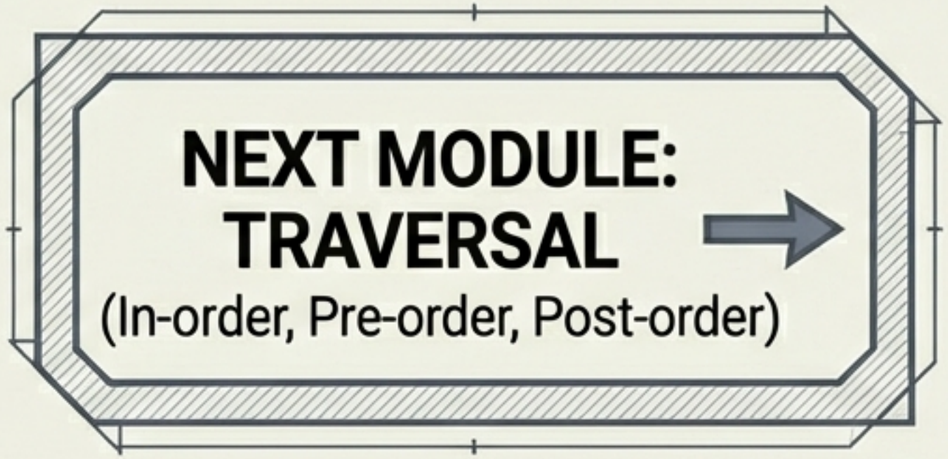
Linked List Representation: The Industry Standard



- **Dynamic**: Memory allocated only when needed.
- **Flexible**: Nodes connect via addresses, not strict positioning.
- **Implementation:** The standard way to code trees in Python, Java, and C++.

Summary & Next Steps

- **Anatomy:** Root, Internal, Leaf
- **Logic:** Inverted Tree, No Sibling Access
- **Types:** BST, Full, Complete
- **Storage:** Linked Lists > Arrays



You now understand the structure.
Next, we learn how to walk the path.